

GOVERNANCE DES DONNÉES

Analyse de l'existant & plan de gouvernance

Pipeline médaillon NYC TLC · AWS · eu-west-3

M2 Data Engineering & Intelligence Artificielle - EFREI

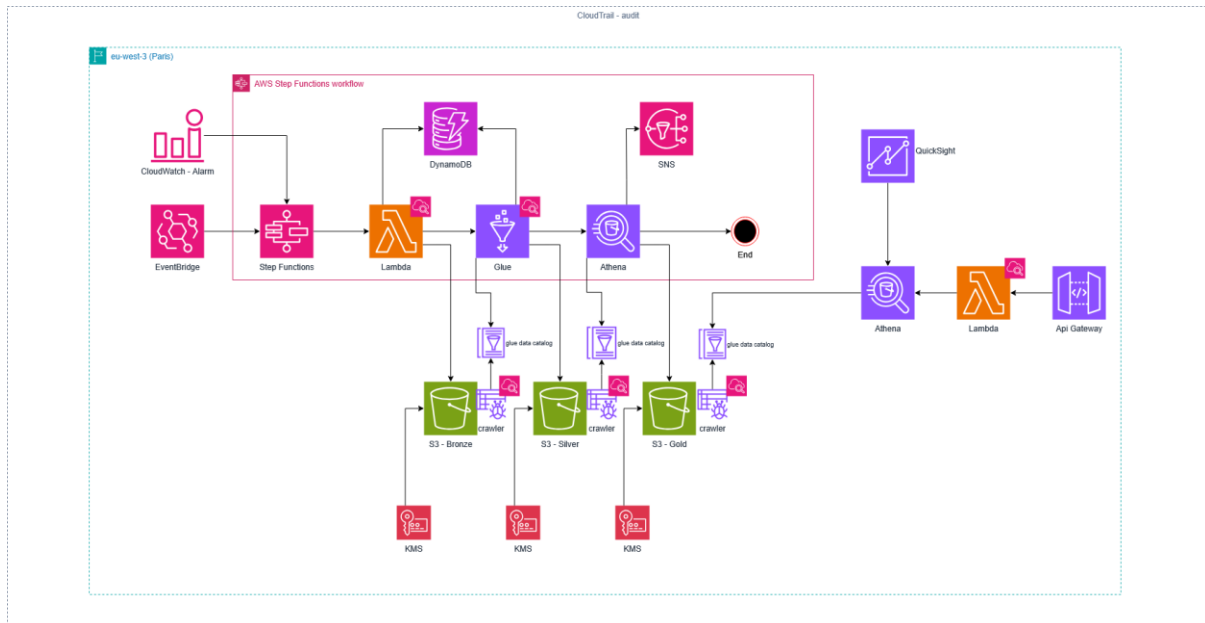
Jacques Lin · Thomas Coutarel

Sommaire

Sommaire.....	1
Analyse de l'existant.....	3
0. Cartographie du projet et des données.....	3
Sources	3
Types de données	3
Sensibilité et data mapping	3
1. Qualité des données.....	4
2. Sécurité et gestion des accès	4
3. Traçabilité (data lineage)	4
4. Documentation	5
Synthèse	5
Plan de gouvernance des données	6
1. Introduction	6
2. Vision de la gouvernance.....	6
3. Rôles et responsabilités.....	6
4. Processus de gouvernance (cycle de vie de la donnée)	6
5. Normes et standards de données	7
Registre de traitement (simplifié)	7
Rétention par couche	7
Dictionnaire de données (extrait)	8
6. Outils de gouvernance	8
7. Plan de mise en œuvre	8
8. Suivi et évaluation	9
9. Risques et mesures.....	9
Schéma d'architecture cible	10
Lecture du schéma	10
Implémentation technique simplifiée	12
4.1 Contrôle qualité.....	12
4.2 Transformation et protection RGPD.....	12
4.3 Journalisation des opérations (lineage).....	12
4.4 Gestion des accès	13
4.5 Déploiement et organisation	13
Explication et justification de la solution.....	14
Justification des choix techniques.....	14
Annexe A. Script complet	14

PARTIE I

Analyse de l'existant



Notre pipeline médaille (bronze, puis silver, puis gold) a d'abord été conçue pour répondre à deux objectifs : fonctionner de bout en bout et limiter les coûts. Ces deux objectifs sont atteints, mais ils ne couvrent pas la gouvernance des données, qui suppose de pouvoir prouver la qualité, suivre les transformations et documenter le sens des données. Cette partie analyse l'existant selon les quatre axes demandés, à savoir la qualité, la sécurité, la traçabilité et la documentation, après une cartographie des données. Nous avons cherché à identifier de façon honnête les points forts du projet et ceux qui restent à améliorer.

0. Cartographie du projet et des données

Sources

Les données proviennent d'une seule source, le CDN officiel de la **NYC Taxi & Limousine Commission** (d37ci6vzurychx.cloudfront.net/trip-data), qui publie des fichiers Parquet mensuels par service. Il n'y a pas d'API temps réel, de flux IoT ni de saisie utilisateur : l'ingestion se fait par fichiers, en batch mensuel. Nous utilisons trois jeux : yellow taxi, green taxi et fhvvhv (VTC à haut volume, de type Uber ou Lyft). La source est publique, mais cela ne garantit pas l'absence de données personnelles, ce que nous avons voulu vérifier.

Types de données

Les données sont **structurées et tabulaires** (format Parquet, schéma stable et typé). Il n'y a pas de données non structurées comme du texte libre, des images ou des logs. Chaque ligne correspond à un trajet, avec des colonnes temporelles (pickup et dropoff à la seconde), spatiales (PULocationID et DOLocationID), économiques (fare_amount, tip_amount, driver_pay) et, pour le seul fhvvhv, des identifiants de base et de licence (hvfhs_license_num, dispatching_base_num, originating_base_num).

Sensibilité et data mapping

Le jeu de données est déjà **partiellement anonymisé à la source** : depuis 2016, la TLC ne publie plus les coordonnées GPS précises, remplacées par environ 260 zones. Il n'y a donc pas de donnée directement identifiante, mais quelques identifiants indirects et des quasi-identifiants subsistent. Par ailleurs, aucun

outil ne vérifie automatiquement l'absence de données personnelles résiduelles, puisque Macie n'est pas déployé. Le tableau ci-dessous récapitule notre inventaire.

Champ ou catégorie	Type d'identification	Risque	Traitement actuel
Identité directe (nom, email, téléphone)	Directe	Sans objet	Absente à la source, jamais publiée par la TLC
Identifiants fhvhv (licence, base)	Indirecte	Moyen	Pseudonymisation par hachage SHA-256 salé (pepper), sans table de correspondance
PULocationID et DOLocationID	Localisation	Moyen	Déjà généralisé en zones (environ 260) à la source
pickup et dropoff datetime (seconde)	Quasi-identifiant	Moyen	Aucun, la valeur est conservée à la seconde
Zone croisée avec l'horodatage précis	Quasi-identifiant combiné	Moyen-Élevé	Non traité, le risque n'est pas encore évalué
Données financières (fare, tip, driver_pay)	Économique	Faible	Agrégées en gold
Données de l'article 9 (santé, opinion, origine)	Sensible	Sans objet	Absentes du jeu de données

L'inventaire montre que le seul traitement appliqué, le hachage des identifiants fhvhv, ne couvre pas le principal risque restant, à savoir une ré-identification possible en croisant la zone et l'horodatage précis. Ce point n'est pas encore évalué et fait l'objet d'une mesure dans le plan de gouvernance.

1. Qualité des données

Ce qui existe : une étape de contrôle qualité (Glue Data Quality, en langage DQDL) en couche silver, un schéma canonique commun aux trois services et une déduplication dans le job Glue.

Points à améliorer :

- **Contrôle à un seul endroit.** La qualité n'est vérifiée qu'en silver. L'ingestion bronze ne contrôle pas la conformité des fichiers (types, colonnes attendues, plages de valeurs), et la couche gold n'a pas de test sur les agrégats produits.
- **Résultats non suivis.** Les résultats de l'étape qualité ne sont pas publiés vers CloudWatch ni conservés. Nous ne pouvons donc pas suivre l'évolution de la qualité dans le temps.
- **Pas de quarantaine.** Le devenir des lignes qui échouent au contrôle n'est pas défini, qu'il s'agisse d'un rejet ou d'un blocage.
- **Critères non formalisés.** Les dimensions de qualité (complétude, unicité, validité, cohérence, exactitude, fraîcheur) ne sont écrites nulle part.

2. Sécurité et gestion des accès

Ce qui existe : des accès en IAM ABAC (aws:PrincipalTag), un chiffrement SSE-KMS (CMK) sur bronze et silver et SSE-S3 sur gold, les Bucket Keys activés, CloudTrail actif, et une API protégée par clé et Usage Plan, avec des ExecutionParameters contre l'injection SQL.

Points à améliorer :

- **Sensibilité déclarée mais non vérifiée.** Le tag DataSensitivity est renseigné à la main et Macie n'est pas déployé. Nous supposons qu'il ne reste pas d'identifiant après hachage, sans l'avoir prouvé avec un outil.

- **Clés KMS sans cycle de vie.** Il n'y a pas de rotation des clés ni de procédure de révocation documentée. Le chiffrement est en place, mais sa gestion dans le temps ne l'est pas.
- **Accès non audités.** Aucune revue des accès n'est réalisée, IAM Access Analyzer n'étant pas utilisé. Le modèle ABAC est bien construit, mais nous ne l'avons pas contrôlé.
- **Exposition de l'API.** L'accès à l'API repose sur une clé associée à un Usage Plan, ce qui n'est pas une authentification au sens strict, et le CORS est ouvert. Nous l'assumons comme un compromis, à documenter.

3. Traçabilité (data lineage)

Ce qui existe : CloudTrail (accès aux API AWS), un registre DynamoDB (avancement du pipeline), l'historique des exécutions Step Functions et le Glue Data Catalog (schémas).

Points à améliorer :

- **Pas de lineage métier.** Nous ne pouvons pas relier une colonne de gold (par exemple `revenu_par_min`) à ses colonnes d'origine en bronze et aux transformations appliquées. Aucun outil de lineage (Glue lineage, DataZone, OpenLineage) n'est en place.
- **CloudTrail suit les accès, pas les transformations.** Il enregistre les appels aux API AWS, donc qui accède à quoi, mais pas ce que devient la donnée.
- **Pas de versioning S3 sur bronze.** Si un fichier source est réécrit, l'état précédent est perdu et nous n'avons pas d'historique du brut.
- **Transformations non versionnées.** On ne peut pas relier un lot produit à la version du code Glue qui l'a généré.

4. Documentation

Ce qui existe : un rapport technique en cours, un dépôt Git (CloudFormation), et le Glue Data Catalog qui documente le schéma technique.

Points à améliorer :

- **Pas de dictionnaire de données métier.** Le catalog donne le type des colonnes (par exemple `tprep_pickup_datetime` en timestamp), mais pas leur définition métier, leur unité, leurs valeurs possibles ni leur propriétaire.
- **Rôles non définis.** Aucun rôle de gouvernance n'est attribué (Data Owner, Data Steward, Data Users) et la responsabilité des données n'est portée par personne d'identifié.
- **Registre RGPD non rédigé.** Il est prévu, mais pas encore écrit.
- **Traitements documentés uniquement dans le code.** La description des pipelines n'est accessible qu'en lisant le code source.

Synthèse

La lecture des quatre axes fait apparaître un écart : la sécurité est assez avancée sur ses aspects techniques, alors que la traçabilité et la documentation métier sont encore peu développées.

Axe	Niveau actuel	Principal point à améliorer	Priorité
Qualité	Moyen	Contrôle seulement en silver, non suivi, pas de quarantaine	Moyenne
Sécurité	Assez bon	Sensibilité non vérifiée, clés sans rotation, accès non audités	Moyenne

Axe	Niveau actuel	Principal point à améliorer	Priorité
Traçabilité	Faible	Pas de lineage métier, pas de versioning S3 sur bronze	Élevée
Documentation	Faible	Pas de dictionnaire de données, aucun rôle défini	Moyenne-Élevée

En résumé. Notre projet livre une pipeline qui fonctionne et reste économe, mais la donnée n'est pas encore suffisamment tracée, documentée dans son sens, ni contrôlée de façon vérifiable. Les mécanismes de gouvernance existent surtout sous forme de briques techniques isolées. Le plan de gouvernance présenté dans la partie suivante vise à combler ces manques.

PARTIE II

Plan de gouvernance des données

1. Introduction

Contexte. Notre pipeline médaillon traite les données de mobilité NYC TLC sur AWS. Elle a été construite pour fonctionner de bout en bout et pour rester peu coûteuse, ce qui est aujourd'hui le cas. En revanche, la donnée n'est pas encore suivie dans ses transformations, ni documentée dans son sens, ni contrôlée de façon vérifiable, et la gouvernance se limite pour l'instant à des briques techniques isolées (chiffrement, ABAC, une étape qualité). Ce plan a pour but de les organiser en une démarche cohérente.

Objectifs. Faire évoluer la pipeline d'un état fonctionnel vers un état fiable et documenté : garantir une qualité mesurable et suivie, appliquer le RGPD couche par couche, rendre les transformations traçables, attribuer des responsabilités claires, et documenter le sens des données autant que leur schéma technique.

Champ d'application. Les trois jeux TLC (yellow, green, fhvvhv), les trois couches S3 (nyctlc-s3-bronze, silver et gold), l'API REST d'exposition et le dashboard QuickSight.

2. Vision de la gouvernance

Définition. La gouvernance des données regroupe les rôles, les règles, les processus et les outils qui permettent de garantir que les données sont fiables, sécurisées, traçables et documentées tout au long de leur cycle de vie.

Objectifs. Les indicateurs exposés en gold, à savoir le revenu par zone et par heure (P1), la comparaison des services (P2) et la dynamique de la demande par jour (P3), doivent pouvoir être expliqués. Il faut pouvoir dire d'où ils viennent, par quelles transformations et avec quel niveau de qualité.

Alignement avec le projet. La gouvernance vient fiabiliser les sources des trois problématiques métier, tout en respectant la contrainte de coût du projet. Nous privilégions des outils dimensionnés au juste besoin, par exemple un scan Macie ponctuel plutôt qu'en continu, ou un lineage documenté plutôt qu'un service dédié facturé.

3. Rôles et responsabilités

L'équipe compte trois membres. Nous avons réparti les rôles de gouvernance de la façon suivante.

Rôle	Titulaire	Responsabilité
Data Governance Officer	Jacques Lin	Règles transverses (RGPD, accès, rétention, gestion des clés) et arbitrages entre sécurité et coût.
Data Steward	Thomas Coutarel	Qualité au quotidien : règles DQ (DQDL), suivi des métriques qualité, gestion de la quarantaine.
Data Users	Équipe et examinateur	Consommateurs du gold via l'API et QuickSight, tenus de respecter les règles d'usage.

Organisation. Le comité de gouvernance correspond à l'équipe projet réunie régulièrement pour valider les règles et passer en revue les incidents de qualité et d'accès. À l'échelle du projet, une organisation à trois rôles est suffisante et une structure plus lourde ne serait pas justifiée.

4. Processus de gouvernance (cycle de vie de la donnée)

Nous distinguons cinq étapes, qui correspondent au cycle de vie de la donnée et servent de trame au schéma d'architecture cible.

- **Ingestion (bronze).** Batch mensuel depuis le CDN TLC. À ajouter : un contrôle de conformité à l'entrée portant sur les colonnes attendues, les types et la présence du fichier.
- **Qualité (silver).** Étape EvaluateDataQuality. À renforcer : la publication des métriques vers CloudWatch, une stratégie de quarantaine, et un ordre consistant à publier les résultats avant de lever une erreur.
- **Sécurité (transverse).** Chiffrement KMS et S3 et ABAC déjà en place. À ajouter : la rotation des clés, la vérification de la sensibilité (Macie ponctuel) et la revue des accès (IAM Access Analyzer).
- **Rétention et cycle de vie.** Règles lifecycle S3 par couche. À ajouter : le versioning S3 sur bronze et une procédure de suppression (droit à l'effacement).
- **Accès et partage (gold).** API par clé et Usage Plan, dashboard QuickSight. À documenter : le moindre privilège, les logs d'accès et l'absence de ré-identification par croisement de filtres.

5. Normes et standards de données

Qualité, avec six dimensions et leurs seuils, ce qui constitue le contrat de données qui manque aujourd'hui :

Dimension	Règle (exemple TLC)	Seuil
Complétude	PULocationID et pickup_datetime non nuls	100 %
Validité	PULocationID entre 1 et 265, montants positifs ou nuls	au moins 99 %
Unicité	Pas de trajet dupliqué (clé métier)	100 %
Cohérence	dropoff_datetime postérieur ou égal à pickup_datetime	au moins 99,5 %
Exactitude	Durée de trajet plausible, inférieure à 24 heures	au moins 99 %
Fraîcheur	Mois attendu présent après publication TLC	oui ou non

Méthode. Glue Data Quality (DQDL) intégré au job silver, résultats publiés vers CloudWatch, blocage ou quarantaine selon la règle.

Conformité. Le RGPD s'applique par couche (registre de traitement, rétention par couche, droit à l'effacement, gestion des clés). Il n'y a ni HIPAA (aucune donnée de santé) ni donnée de l'article 9, si bien qu'une analyse d'impact (AIPD) n'est pas requise. Les identifiants indirects sont pseudonymisés par hachage salé. Comme le hachage reste, sur le plan juridique, une pseudonymisation et non une anonymisation complète, ces valeurs demeurent des données à caractère personnel au sens du RGPD, ce qui justifie les règles de rétention et le registre présentés ci-dessous.

Format et nommage. Schéma canonique commun aux trois services, convention nyctlc-* et tags (Project, Layer, DataSensitivity), et un dictionnaire de données dont un extrait figure plus bas.

Registre de traitement (simplifié)

Finalité	Base légale	Catégories de données	Rétention
Produire des indicateurs agrégés de mobilité urbaine (revenu par zone et heure, comparaison des services, dynamique de la demande)	Intérêt légitime, pour une finalité statistique, sur des données déjà dépourvues d'identifiant direct à la source	Données de trajets (horodatage, zones, montants), et identifiants indirects de bases et de licences, pseudonymisés	Voir la rétention par couche ci-dessous

Rétention par couche

- **Bronze** : 12 mois, avec versioning, puis suppression automatique par règle lifecycle.
- **Silver** : 12 mois.

- **Gold** : 24 mois, la rétention plus longue étant justifiée par des agrégats sans identifiant.
- **Quarantaine** : 30 jours, le temps d'analyser les rejets, puis suppression.
- **Résultats de requêtes de l'API (Athena)** : 7 jours.

Droit à l'effacement. Les données étant pseudonymisées puis agrégées, et aucune table de correspondance ne permettant de relier un enregistrement à une personne précise, une demande individuelle ne peut pas viser une ligne donnée. La protection repose donc sur la pseudonymisation et l'agrégation, ce qui rejoint le cas prévu à l'article 11 du RGPD, relatif au traitement ne nécessitant pas l'identification de la personne.

Dictionnaire de données (extrait)

Champ	Couche	Type	Définition métier	Domaine ou unité	Sensibilité
pickup_datetime	silver	timestamp	Date et heure de prise en charge	horodatage	quasi-identifiant
pickup_hour	silver	timestamp	Heure de prise en charge généralisée	arrondi à l'heure	dérivé, k-anonymat
PULocationID	bronze, silver	entier	Zone de prise en charge	1 à 265	localisation
DOLocationID	bronze, silver	entier	Zone de dépose	1 à 265	localisation
fare_amount	bronze, silver	décimal	Montant de la course	dollars, positif	économique
hvfhs_license_num	silver	chaîne hachée	Licence de l'opérateur VTC, pseudonymisée	valeur hachée	identifiant indirect
dispatching_base_num	silver	chaîne hachée	Base de répartition, pseudonymisée	valeur hachée	identifiant indirect

6. Outils de gouvernance

Pour chaque besoin, nous avons retenu un service AWS en tenant compte du coût.

Besoin	Outil retenu	Raison (coût)
Qualité	AWS Glue Data Quality (DQDL)	Intégré au job silver, aucun outil tiers à héberger.
Sécurité et accès	IAM ABAC, KMS, Macie (ponctuel), Access Analyzer	Access Analyzer gratuit, Macie lancé ponctuellement plutôt qu'en continu.
Traçabilité	CloudTrail, versioning S3, lineage documenté	DataZone et OpenLineage écartés pour le coût, ce choix étant expliqué.
Documentation	Glue Data Catalog, dictionnaire métier	Catalog déjà en place, dictionnaire sans coût.
Monitoring et alerting	CloudWatch, SNS	Déjà en place, étendus aux métriques qualité.

Automatisation. Les contrôles s'intègrent dans la state machine Step Functions existante, et les métriques qualité sont publiées à chaque exécution.

7. Plan de mise en œuvre

- **Phase 1, audit** : réalisée, avec l'analyse de l'existant et la cartographie.
- **Phase 2, règles** : le présent plan.
- **Phase 3, mise en œuvre technique** : script de contrôle qualité, journalisation des opérations et gestion des accès.
- **Phase 4, documentation** : dictionnaire de données, registre RGPD et schéma annoté.

Ressources. Une équipe de trois personnes, les crédits AWS Educate et des services serverless dont le coût reste très faible en dehors des exécutions.

8. Suivi et évaluation

- **Indicateurs.** Taux de conformité qualité par dimension, part des champs couverts par le dictionnaire, nombre d'accès non conformes détectés, et fraîcheur mesurée par la présence du mois attendu.
- **Contrôles.** Revue régulière de CloudTrail, et alarmes CloudWatch sur les échecs qualité comme sur les problèmes d'infrastructure, déjà en place côté Step Functions.
- **Amélioration continue.** Chaque incident de qualité ou d'accès sert à revoir les règles DQDL et les droits IAM.

9. Risques et mesures

Risque	Niveau	Mesure prévue
Ré-identification en croisant la zone et l'horodatage précis	Élevé	Généralisation temporelle (heure au lieu de la seconde) ou test de k-anonymat (k supérieur ou égal à 5).
Perte du brut, faute de versioning S3 sur bronze	Élevé	Activer le versioning S3 et une politique de rétention.
Qualité qui se dégrade sans être détectée	Moyen	Publier les métriques qualité vers CloudWatch et poser des alarmes.
Sensibilité non vérifiée, Macie absent	Moyen	Lancer un scan Macie ponctuel pour vérifier l'absence de PII.
Coût trop élevé lié à un sur-outillage	Moyen	Rester sur des services natifs et serverless, avec des scans à la demande.

Conclusion. Ce plan organise les briques techniques déjà présentes en une démarche de gouvernance plus complète, structurée autour des quatre axes (qualité, sécurité, traçabilité, documentation) et de la conformité RGPD par couche. Il sert de base à la conception de l'architecture cible et à la mise en œuvre technique décrites dans la suite du dossier.

PARTIE III

Schéma d'architecture cible

Le schéma ci-dessous représente le cycle de vie complet de la donnée, de l'ingestion depuis la source TLC jusqu'à l'exposition via l'API et le dashboard. Il fait apparaître les trois zones de stockage médaillon (brut, nettoyé, prêt à l'analyse) et les mécanismes de contrôle appliqués à chaque couche. Les contrôles déjà présents dans le projet initial sont distingués des ajouts de gouvernance, préfixés par le signe plus, afin de visualiser l'apport de cette démarche. Les techniques de protection RGPD sont indiquées à l'endroit où elles s'appliquent.

Architecture cible — Pipeline médaillon gouvernée

Cycle de vie de la donnée · zones de stockage · mécanismes de contrôle (NYC TLC / AWS eu-west-3)

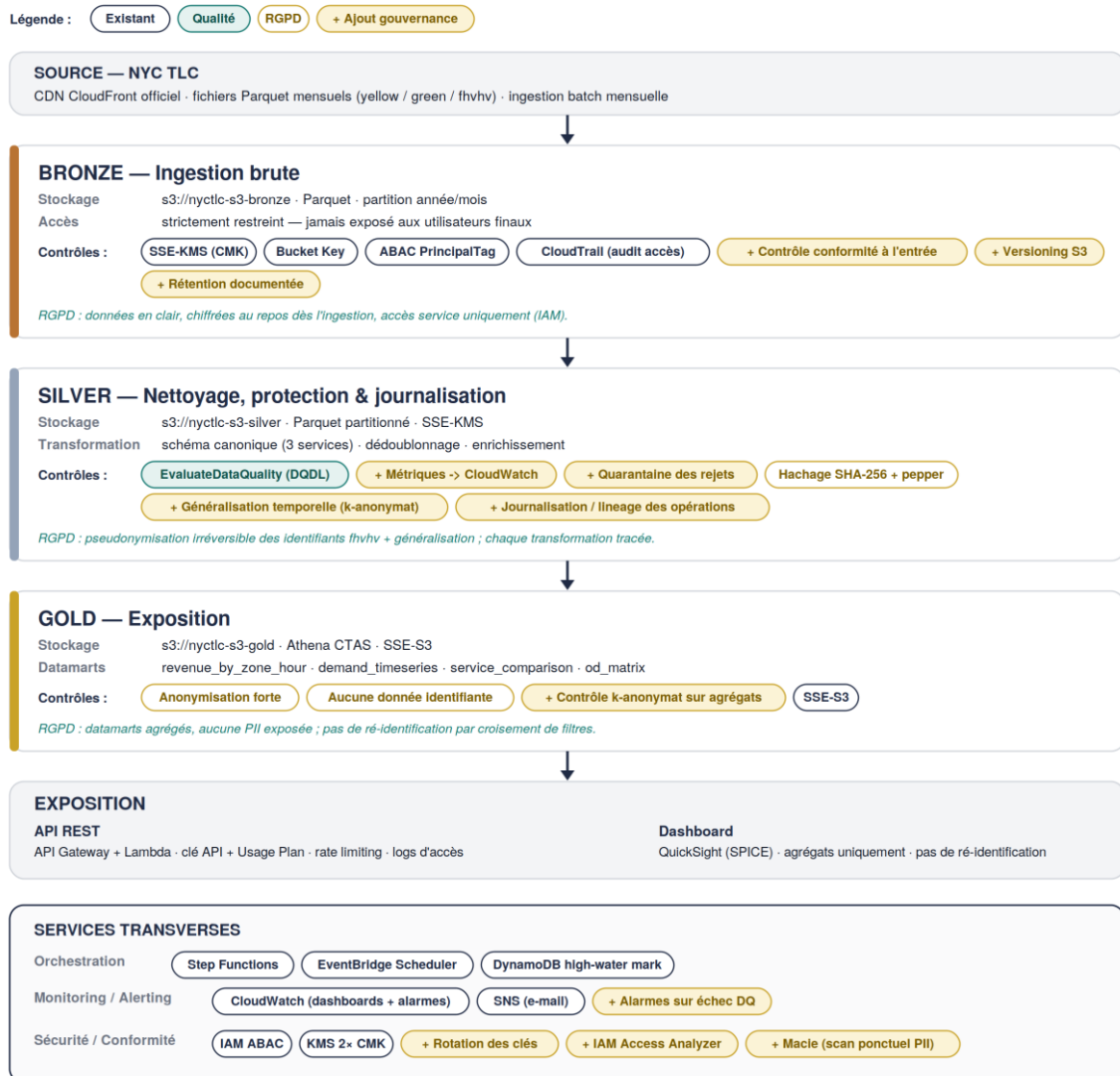


Figure 1. Architecture médaillon gouvernée : zones de stockage, flux et mécanismes de contrôle.

Lecture du schéma

- **Bronze** : la donnée brute est chiffrée au repos dès l'ingestion (SSE-KMS) et son accès est strictement restreint (ABAC, jamais exposée). Les ajouts de gouvernance renforcent l'entrée avec un contrôle de conformité, le versioning S3 et une rétention documentée.

- **Silver** : c'est la couche pivot de la gouvernance. L'étape qualité (DQDL) devient observable grâce aux métriques CloudWatch, et protectrice grâce à la quarantaine. Les identifiants indirects sont pseudonymisés (SHA-256 et pepper) et les horodatages sont généralisés (k-anonymat). Enfin, chaque exécution produit un enregistrement de journalisation.
- **Gold** : les datamarts sont agrégés et ne contiennent aucune donnée identifiante, avec un contrôle de k-anonymat sur les agrégats avant exposition.
- **Exposition et services transverses** : l'API (clé, Usage Plan, rate limiting, logs) et le dashboard n'exposent que des agrégats. Les services transverses ajoutent la rotation des clés KMS, IAM Access Analyzer et un scan Macie ponctuel.

PARTIE IV

Implémentation technique simplifiée

La démarche est illustrée par un job AWS Glue (Spark) qui met en œuvre la couche silver gouvernée, le fichier `nyctlc_glue_governance_silver.py`, fourni en Annexe A. Ce script simule un traitement complet intégrant les quatre mécanismes demandés, à savoir le contrôle qualité, la transformation, la journalisation des opérations et la gestion des accès, sur la période de test 2025-01. Les extraits ci-dessous en présentent le cœur.

4.1 Contrôle qualité

Un contrat de données explicite (ruleset DQDL) est évalué par le nœud `EvaluateDataQuality`. Les métriques sont publiées vers CloudWatch et le détail est archivé dans S3. Les lignes temporellement incohérentes sont dirigées vers une zone de quarantaine plutôt que rejetées sans trace. En cas d'échec, l'écriture vers gold est bloquée et une alerte SNS est émise.

Extrait : ruleset DQDL et publication des résultats

```
DQ_RULESET = """
Rules = [
    Completeness "PULocationID" = 1.0,
    Completeness "pickup_datetime" = 1.0,
    ColumnValues "PULocationID" between 1 and 265,
    ColumnValues "fare_amount" >= 0,
    ColumnValues "trip_distance" >= 0
]"""

dq = EvaluateDataQuality().process_rows(frame=df, ruleset=DQ_RULESET,
    publishing_options={
        "enableDataQualityCloudWatchMetrics": True, # vers CloudWatch
        "enableDataQualityResultsPublishing": True,
        "resultsS3Prefix": ARGS["dq_results_path"] }) # archive S3
```

Le contrôle qualité devient ainsi contractualisé, historisé dans CloudWatch et doté d'une gestion explicite des rejets, ce qui répond aux trois manques relevés dans l'analyse.

4.2 Transformation et protection RGPD

Deux protections RGPD sont appliquées lors du passage de bronze à silver. La pseudonymisation par hachage SHA-256 salé (pepper) protège les identifiants indirects. La généralisation temporelle, qui tronque l'horodatage à l'heure, réduit le risque de ré-identification par croisement de la zone et du temps, identifié comme le risque principal.

Extrait : pseudonymisation et généralisation temporelle

```
def sha256_with_pepper(pepper):
    def _h(v):
        if v is None: return None
        return hashlib.sha256((pepper + str(v)).encode()).hexdigest()
    return F.udf(_h, StringType())

# généralisation temporelle : la précision seconde est un quasi-identifiant
coherent = coherent.withColumn(
    "pickup_hour", F.date_trunc("hour", F.col("pickup_datetime")))
```

4.3 Journalisation des opérations (lineage)

Cette étape répond au point le plus faible relevé dans l'analyse, la traçabilité. À chaque exécution, un enregistrement d'audit est écrit dans une table de gouvernance dédiée. Il indique qui a exécuté le traitement (principal IAM), d'où vient la donnée et où elle va, combien de lignes ont été traitées ou mises

en quarantaine, le score de qualité obtenu, et quelles techniques RGPD ont été appliquées. La donnée elle-même est ainsi tracée, et pas seulement l'infrastructure.

Extrait : enregistrement de journalisation et de lineage

```
audit = {
  "run_id": RUN_ID,          "principal": CALLER,      # qui
  "source": bronze_path,    "target": silver_path,    # où
  "rows_in": rows_in,       "rows_valid": rows_valid,
  "rows_quarantined": rows_rejected,
  "dq_status": dq_status,   "dq_score": dq_score,      # qualité
  "rgpd_transformations": applied_rgpd }          # protections
write_audit_record(audit) # table d'audit S3 partitionnée
```

4.4 Gestion des accès

- **Secret externalisé.** Le pepper n'est jamais en dur dans le code. Il est lu depuis AWS Secrets Manager, dont l'accès est restreint au seul rôle du job Glue par une policy IAM.
- **Rôle ABAC.** Le job s'exécute sous un rôle taggé, et l'accès aux buckets est régi par les politiques ABAC (aws:PrincipalTag) déjà en place.
- **Quarantaine restreinte.** Les lignes rejetées sont écrites dans un préfixe séparé à accès limité, jamais propagées vers gold.
- **Alerte.** Un échec qualité déclenche une notification SNS par e-mail, et l'incident est visible immédiatement.

4.5 Déploiement et organisation

Conformément aux attendus (stockage des fichiers, exécution des scripts, organisation des dossiers) :

- **Stockage des fichiers.** Buckets S3 par couche (nyctlc-s3-bronze, silver, gold). Les scripts et les sorties de gouvernance sont dans nyctlc-s3-scripts, sous les préfixes dq-results, governance-audit et _quarantine.
- **Exécution des scripts.** Le job Glue est déclenché dans la state machine Step Functions existante, avec les paramètres passés à l'exécution (service, période, chemins, secret, topic SNS).
- **Organisation des dossiers (dépôt Git).** Regroupement par domaine logique en storage, security, compute, orchestration, api et observability. Le script de gouvernance et son registre d'audit relèvent de compute et d'observability.

Le script complet figure en Annexe A et se déploie tel quel sur Glue 4.0.

PARTIE V

Explication et justification de la solution

Le tableau ci-dessous résume l'amélioration apportée par la démarche de gouvernance, axe par axe, par rapport à la situation initiale.

Axe	Situation initiale	Après gouvernance
Qualité	Étape DQ unique en silver, non observable, pas de quarantaine	Contrat DQDL, métriques CloudWatch, quarantaine, blocage et alerte SNS
Sécurité	Chiffrement et ABAC, mais secret potentiel en clair et pas de rotation	Pepper dans Secrets Manager, rotation KMS, Access Analyzer, Macie ponctuel
Traçabilité	Aucun lineage métier, on trace l'infrastructure et pas la donnée	Journalisation par exécution : principal, volumes, score DQ, protections RGPD
Documentation	Schéma technique seul, aucun rôle défini	Rôles attribués, dictionnaire de données, registre RGPD, schéma annoté
RGPD	Hachage des seuls identifiants fhvvh	Hachage, généralisation temporelle (k-anonymat) et contrôle des agrégats gold

Justification des choix techniques

- **Hachage SHA-256 salé plutôt que tokenisation réversible.** Le besoin analytique ne demande aucune ré-identification. L'absence de table de correspondance et le pepper secret rendent la valeur difficile à inverser en pratique, tout en restant une pseudonymisation au sens juridique.
- **Généralisation temporelle plutôt que suppression de l'horodatage.** Conserver l'heure préserve la valeur analytique (P1 et P3) tout en réduisant le quasi-identifiant présent à la seconde.
- **Journalisation propre plutôt que DataZone ou OpenLineage.** Une table d'audit S3 partitionnée couvre le besoin de lineage à faible coût, en cohérence avec la contrainte FinOps. Un service de catalogue dédié serait sur-dimensionné à cette échelle.
- **Glue Data Quality natif plutôt que Great Expectations ou Soda.** L'intégration est directe dans le job existant, la publication CloudWatch est native, et aucun composant tiers n'est à héberger ni à facturer.
- **Quarantaine plutôt que rejet systématique.** Les lignes incohérentes sont isolées et conservées pour analyse, sans interrompre le traitement des lignes valides.

Conclusion. La solution ne réécrit pas la pipeline, elle l'instrumente. En ajoutant une étape qualité observable, deux protections RGPD ciblées sur le risque réel, une journalisation de bout en bout et une gestion rigoureuse des secrets, elle rend la pipeline plus fiable et vérifiable, tout en respectant la contrainte de coût du projet. Chaque mécanisme répond à un point faible précis relevé dans l'analyse de l'existant.

Annexe A. Script complet

Job AWS Glue (Spark) **nyctlc_glue_governance_silver.py**, couche silver gouvernée, déployable tel quel sur Glue 4.0.

```
"""
=====
nyctlc_glue_governance_silver.py
=====
Job AWS Glue (Spark). Couche SILVER gouvernée du pipeline médaillon NYC TLC.

Illustre les quatre mécanismes de gouvernance exigés par le TP :
1. CONTROLE QUALITE : EvaluateDataQuality (DQDL), métriques CloudWatch,
archivage S3 des résultats et quarantaine des rejets.
2. TRANSFORMATION : schéma canonique, dédoublement,
pseudonymisation SHA-256 + pepper (RGPD),
généralisation temporelle (k-anonymat).
3. JOURNALISATION : écriture d'un enregistrement de lineage par exécution
(qui, quoi, combien, quand) dans une table d'audit.
4. GESTION DES ACCES : secret (pepper) lu depuis Secrets Manager, rôle Glue
ABAC, quarantaine écrite dans un préfixe restreint,
alerte SNS en cas d'échec qualité.

Déploiement : job Glue 4.0 (Spark). Paramètres passés via --KEY value.
Cible testée : période 2025-01 (publication TLC stable).
=====
"""

import sys
import json
import uuid
import hashlib
import datetime as dt

import boto3
from aws glue.transforms import *
from aws glue.utils import getResolvedOptions
from aws glue.context import GlueContext
from aws glue.job import Job
from aws glue.dynamicframe import DynamicFrame
from aws glue.dq.transforms import EvaluateDataQuality # module DQ intégré à Glue

from pyspark.context import SparkContext
from pyspark.sql import functions as F
from pyspark.sql.types import StringType

# ----- #
# 0. PARAMÈTRES & CONTEXTE
# ----- #
ARGS = getResolvedOptions(sys.argv, [
    "JOB_NAME",
    "service_type", # yellow | green | fhvhv
    "period", # ex. 2025-01
    "bronze_path", # s3://nyctlc-s3-bronze/<service>/<period>/
    "silver_path", # s3://nyctlc-s3-silver/<service>/
    "quarantine_path", # s3://nyctlc-s3-silver/_quarantine/<service>/
    "dq_results_path", # s3://nyctlc-s3-scripts/dq-results/<service>/
    "audit_path", # s3://nyctlc-s3-scripts/governance-audit/
    "pepper_secret_id", # id du secret Secrets Manager contenant le pepper
    "sns_topic_arn", # ARN du topic SNS d'alerting
    "region", # eu-west-3
])

REGION = ARGS["region"]
RUN_ID = str(uuid.uuid4())
STARTED_AT = dt.datetime.now(dt.timezone.utc)

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(ARGS["JOB_NAME"], ARGS)

cw = boto3.client("cloudwatch", region_name=REGION)
sns = boto3.client("sns", region_name=REGION)
sts = boto3.client("sts", region_name=REGION)
CALLER = sts.get_caller_identity()["Arn"] # principal exécutant (traçabilité)

# ----- #
# 4. GESTION DES ACCES : le pepper n'est JAMAIS en dur dans le code
# Lu depuis Secrets Manager ; l'accès au secret est régi par
# une policy IAM restrictive (seul le rôle du job Glue y accède).
# ----- #
def get_pepper(secret_id: str) -> str:
    client = boto3.client("secretsmanager", region_name=REGION)
    resp = client.get_secret_value(SecretId=secret_id)
    return json.loads(resp["SecretString"])[ "pepper" ]
```



```

# ----- #
# 2. TRANSFORMATION : pseudonymisation RGPD (SHA-256 + pepper)
#   Pseudonymisation : aucune table de correspondance. Le pepper rend
#   l'attaque par dictionnaire/rainbow table impraticable.
# ----- #
def sha256_with_pepper(pepper: str):
    def _h(value):
        if value is None:
            return None
        return hashlib.sha256((pepper + str(value)).encode("utf-8")).hexdigest()
    return F.udf(_h, StringType())

# Identifiants indirects à pseudonymiser, par service
PII_INDIRECT = {
    "fhvhv": ["hvfhs_license_num", "dispatching_base_num", "originating_base_num"],
    "yellow": [],
    "green": [],
}

# Colonnes canoniques temporelles (nom aligné entre services en amont)
PICKUP_COL = "pickup_datetime"
DROPOFF_COL = "dropoff_datetime"

# ----- #
# 1. CONTROLE QUALITE : règles DQDL (contrat de données)
#   Publication CloudWatch + archivage S3 + comportement bloquant.
# ----- #
DQ_RULESET = """
Rules = [
    Completeness "PULocationID" = 1.0,
    Completeness "%s" = 1.0,
    ColumnValues "PULocationID" between 1 and 265,
    ColumnValues "DOLocationID" between 1 and 265,
    ColumnValues "fare_amount" >= 0,
    ColumnValues "trip_distance" >= 0
]
""" % PICKUP_COL

def run_quality_gate(dyf: DynamicFrame):
    """Évalue le ruleset, publie les métriques dans CloudWatch et archive
    le détail (ruleOutcomes) dans S3. Renvoie (statut_global, score, résultats)."""
    dq_results = EvaluateDataQuality().process_rows(
        frame=dyf,
        ruleset=DQ_RULESET,
        publishing_options={
            "dataQualityEvaluationContext": "silver_%s" % ARGS["service_type"],
            "enableDataQualityCloudWatchMetrics": True, # vers CloudWatch
            "enableDataQualityResultsPublishing": True,
            "resultsS3Prefix": ARGS["dq_results_path"], # archivage S3
        },
        additional_options={"performanceTuning.caching": "CACHE_NOTHING"},
    )
    rule_outcomes = SelectFromCollection.apply(dq_results, key="ruleOutcomes")
    outcomes = rule_outcomes.toDF().collect()
    passed = sum(1 for r in outcomes if r["Outcome"] == "Passed")
    total = len(outcomes)
    score = round(passed / total, 4) if total else 0.0
    status = "PASSED" if passed == total else "FAILED"
    return status, score, [(r["Name"], r["Outcome"]) for r in outcomes]

# ----- #
# 3. JOURNALISATION / LINEAGE : un enregistrement d'audit par exécution
#   C'est la brique qui manquait le plus au projet initial : on trace
#   la DONNÉE (qui l'a transformée, combien de lignes, quel score qualité,
#   quelles techniques RGPD appliquées), pas seulement l'infrastructure.
# ----- #
def write_audit_record(record: dict):
    audit_df = spark.createDataFrame([record])
    (audit_df.coalesce(1)
     .write.mode("append")
     .partitionBy("service_type", "period")
     .json(ARGS["audit_path"]))

def notify_failure(record: dict):
    sns.publish(
        TopicArn=ARGS["sns_topic_arn"],
        Subject="[NYC TLC][Gouvernance] Échec contrôle qualité SILVER",
        Message=json.dumps(record, indent=2, default=str),
    )

# ===== #
#                               MAIN
# ===== #
def main():
    service = ARGS["service_type"]
    period = ARGS["period"]

    # --- Lecture BRONZE (données brutes chiffrées au repos) ---

```

```

df = spark.read.parquet(ARGS["bronze_path"])
rows_in = df.count()

# --- Transformation : dédoublonnage + cohérence temporelle ---
df = df.dropDuplicates()
# Séparation des lignes cohérentes / incohérentes (quarantaine)
coherent = df.filter(F.col(DROPOFF_COL) >= F.col(PICKUP_COL))
rejected = df.filter(F.col(DROPOFF_COL) < F.col(PICKUP_COL))
rows_rejected = rejected.count()

# --- RGPD : pseudonymisation des identifiants indirects ---
applied_rgpd = []
pepper = get_pepper(ARGS["pepper_secret_id"])
hash_udf = sha256_with_pepper(pepper)
for col in PII_INDIRECT.get(service, []):
    if col in coherent.columns:
        coherent = coherent.withColumn(col, hash_udf(F.col(col)))
        applied_rgpd.append("sha256_pepper:%s" % col)

# --- RGPD : généralisation temporelle (k-anonymat) ---
# La précision seconde est un quasi-identifiant : on tronque à l'heure
# pour réduire le risque de ré-identification par croisement zone x temps.
coherent = coherent.withColumn(
    "pickup_hour", F.date_trunc("hour", F.col(PICKUP_COL))
)
applied_rgpd.append("generalisation_temporelle:pickup_hour")

# --- CONTRÔLE QUALITÉ (gate DQDL) ---
dyf = DynamicFrame.fromDF(coherent, glueContext, "silver_dyf")
dq_status, dq_score, dq_detail = run_quality_gate(dyf)

rows_valid = coherent.count()

# --- Écriture QUARANTAINE (accès restreint) ---
if rows_rejected > 0:
    (rejected.write.mode("overwrite")
     .parquet("%s%s/" % (ARGS["quarantine_path"], period)))

# --- Enregistrement de JOURNALISATION / lineage ---
audit = {
    "run_id": RUN_ID,
    "service_type": service,
    "period": period,
    "started_at": STARTED_AT.isoformat(),
    "ended_at": dt.datetime.now(dt.timezone.utc).isoformat(),
    "principal": CALLER, # QUI a exécuté (traçabilité)
    "source": ARGS["bronze_path"], # d'OU vient la donnée
    "target": ARGS["silver_path"], # OU elle va
    "rows_in": rows_in,
    "rows_valid": rows_valid,
    "rows_quarantined": rows_rejected,
    "dq_status": dq_status,
    "dq_score": dq_score,
    "dq_rules": [{"rule": n, "outcome": o} for n, o in dq_detail],
    "rgpd_transformations": applied_rgpd, # QUELLES protections appliquées
    "pepper_version": ARGS["pepper_secret_id"],
}
write_audit_record(audit)

# --- Comportement bloquant + alerte si la qualité échoue ---
if dq_status == "FAILED":
    notify_failure(audit)
    raise RuntimeError(
        "Gate qualité SILVER échouée (score=%s), écriture gold bloquée." % dq_score
    )

# --- Écriture SILVER (partitionnée, chiffrée SSE-KMS via config bucket) ---
(coherent.write.mode("overwrite")
 .partitionBy("pickup_hour")
 .parquet("%s%s/" % (ARGS["silver_path"], period)))

print("SILVER OK, run_id=%s dq_score=%s rows_valid=%s quarantine=%s"
      % (RUN_ID, dq_score, rows_valid, rows_rejected))

main()
job.commit()

```